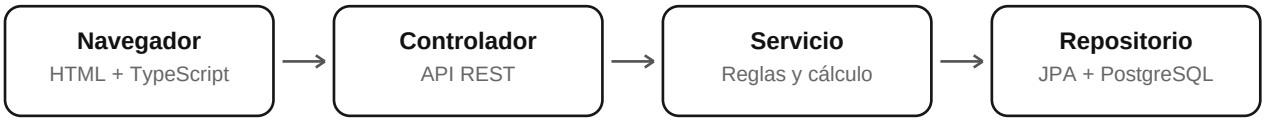
GUÍA DE EXPOSICIÓN

Factura ARCA

CRUD de facturas de venta con Spring Boot, JPA, API REST, TypeScript y HTML5

Respuesta corta para empezar la defensa

Es una aplicación web full stack hecha con Spring Boot. El navegador muestra el frontend, TypeScript llama a una API REST, Spring Boot aplica las reglas de negocio y JPA persiste las facturas y sus detalles en PostgreSQL.



Capa	Tecnología	Responsabilidad
Frontend	HTML5 + CSS + TypeScript	Formularios, tabla dinámica, filtros y acciones.
API	Spring Web	Recibe GET, POST, PUT y DELETE y devuelve JSON/códigos HTTP.
Negocio	@Service + @Transactional	Valida IDs, calcula importes y coordina la escritura.
Datos	Spring Data JPA + PostgreSQL	Guarda cabecera, detalles y relaciones.

Documento preparado para que cualquier integrante pueda explicar el proyecto y ejecutar la demo.

1. Qué se implementó

Se completó el trabajo práctico sobre la base entregada por el grupo. Se mantuvieron el modelo JPA y los reportes anteriores, y se agregó el ciclo de escritura completo de una factura.

- DTOs de entrada que reciben IDs en lugar de entidades anidadas.
- POST para crear una factura con sus renglones en cascada y respuesta 201 Created.
- PUT para actualizar cabecera, ítems o solamente el estado y las observaciones.
- DELETE para eliminación física con respuesta 204 No Content.
- Anulación lógica: estado ANULADA más fechaAnulacion automática.
- Cálculo automático de neto, IVA 21%, subtotal y total.
- Frontend con filtros, alta dinámica, listado, anulación y eliminación.
- Endpoint auxiliar de catálogos para cargar clientes, condiciones, puntos, monedas y artículos reales.
- Manejo uniforme de errores 400, 404 y 409.

Decisión de negocio importante

El proyecto permite las dos acciones que muestra la interfaz: **anular** conserva el comprobante para mantener trazabilidad; **eliminar** lo borra físicamente. El profesor puede preguntar la diferencia: anular cambia estado y fecha, mientras que eliminar ejecuta DELETE sobre la entidad.

¿Es Spring o Spring Boot?

Es **Spring Boot 3**. Spring es el ecosistema y Spring Boot es la forma usada para configurarlo y ejecutarlo rápidamente. La clase **FacturaArcaApplication** usa @SpringBootApplication y levanta Tomcat, la API y el frontend en http://localhost:8080.

Atomicidad

Los métodos de escritura tienen @Transactional. Eso significa que la operación se confirma completa o se revierte completa. Si falla un artículo, no queda una factura sin detalles a mitad de camino.

2. DTOs y cálculo de importes

Un DTO es un objeto pensado para transportar datos. Evita exponer directamente las entidades JPA y define un contrato simple entre frontend y backend.

FacturaRequestDTO

Campo	Tipo Java / TypeScript	Uso
numeroFactura	Long / number	Número visible del comprobante.
fechaEmision	Date / string	Fecha enviada como yyyy-MM-dd.
clienteId	Long / number	Busca el Cliente relacionado.
condicionIvaId	Long / number	Busca la condición fiscal.
puntoVentaId	Long / number	Busca el punto de venta.
tipoMonedaId	Long / number	Busca la moneda.
estado	String / string	EMITIDA, PENDIENTE o ANULADA.
observaciones	String / string	Texto opcional.
detalles	List / array	Renglones de FacturaDetalleRequestDTO.

FacturaDetalleRequestDTO

Cada renglón recibe articuloId, cantidad y porcentajeBonificacion. El precio no se confía al navegador: el backend busca la última ListaPrecioArticulo asociada y usa ese valor oficial.

Ejemplo de cálculo

Paso	Cuenta	Resultado
Base	2 × $1.000	$2.000
Bonificación 10%	$2.000 × 0,90	$1.800 netos
IVA 21%	$1.800 × 0,21	$378
Subtotal	$1.800 + $378	$2.178

Se usa BigDecimal y redondeo HALF_UP a dos decimales para evitar errores típicos de coma flotante durante el cálculo. Luego se asignan los valores al modelo existente.

3. Cómo funciona el backend

Controlador REST

FacturaRestController traduce HTTP a llamadas Java. No contiene cálculos: delega en FacturaService y elige el código de respuesta correcto.

Método y ruta	Respuesta	Qué hace
GET /api/facturas	200 OK	Lista con filtros opcionales.
POST /api/facturas	201 Created	Crea cabecera y detalles; incluye Location.
PUT /api/facturas/{id}	200 OK	Actualiza datos, ítems o estado.
DELETE /api/facturas/{id}	204 No Content	Elimina la factura.
GET /api/catalogos	200 OK	Devuelve opciones reales para los selectores.

Sintaxis esencial

```
@PostMapping
public ResponseEntity<FacturaReporteDTO> crearFactura(
        @RequestBody FacturaRequestDTO request) {
    FacturaReporteDTO creada = facturaService.crear(request);
    return ResponseEntity.created(URI.create("/api/facturas/" + creada.getId()))
            .body(creada);
}
```

- @PostMapping asocia el método Java con una petición HTTP POST.
- @RequestBody transforma el JSON recibido en FacturaRequestDTO.
- ResponseEntity permite definir estado, cabeceras y cuerpo.
- @PathVariable toma el {id} de la URL en PUT y DELETE.
- @RequestParam toma filtros opcionales de la query string.

Servicio y cascada

FacturaService busca cada relación por ID. Si falta, lanza RecursoNoEncontradoException. Después crea cada FacturaVentaDetalle y llama factura.agregarDetalle(detalle), que completa ambos lados de la relación. Como FacturaVenta tiene cascade = CascadeType.ALL, guardar la factura guarda los detalles.

orphanRemoval = true

Al reemplazar los ítems durante un PUT, los detalles que ya no pertenecen a la factura se eliminan de la base. Así la colección y la tabla de detalles permanecen sincronizadas.

4. Cómo funciona el frontend

Spring Boot sirve index.html, styles.css y el JavaScript compilado. El código fuente TypeScript queda en frontend/src y se compila con npm run build.

Las tres secciones pedidas

- Buscar comprobantes: fechas, estado y monto mínimo.
- Nueva factura: cabecera más tabla dinámica de múltiples renglones.
- Comprobantes registrados: listado con acciones Anular y Eliminar.

FacturaService.ts

```
async crearFactura(factura: FacturaRequestDTO): Promise<FacturaReporteDTO> {
  return this.solicitar<FacturaReporteDTO>(this.baseUrl, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(factura),
  });
}
```

- async indica que la función trabaja con una operación asíncrona.
- await espera la respuesta sin bloquear toda la página.
- fetch hace la petición HTTP desde el navegador.
- Promise describe el tipo que llegará en el futuro.
- JSON.stringify convierte el objeto TypeScript a JSON para enviarlo.

app.ts y el DOM

app.ts conserva los ítems en un array en memoria. Cuando el usuario agrega, cambia o quita un renglón, renderizarDetalles vuelve a dibujar la tabla y recalcula la vista previa. Al enviar, crea un FacturaRequestDTO, llama al servicio y vuelve a pedir el listado.

Interfaz

El diseño usa blanco, negro y grises, controles nativos, tipografía del sistema y diseño responsive. No depende de fuentes, frameworks ni recursos externos, por lo que la demo funciona sin internet.

5. Archivos importantes

Archivo / carpeta	Qué contiene
dto/FacturaRequestDTO.java	Cabecera enviada en POST y PUT.
dto/FacturaDetalleRequestDTO.java	Artículo, cantidad y bonificación por renglón.
dto/FacturaReporteDTO.java	Respuesta resumida que consume el frontend.
service/FacturaService.java	Validación, transacción, cálculo, cascada, anulación y eliminación.
controller/FacturaRestController.java	Endpoints GET, POST, PUT y DELETE.
controller/ApiExceptionHandler.java	Convierte excepciones en errores HTTP claros.
repository/*.java	Acceso JPA a facturas y entidades relacionadas.
controller/CatalogoRestController.java	Opciones de selectores para el frontend.
frontend/src/models/factura.dto.ts	Interfaces espejo de los DTOs Java.
frontend/src/FacturaService.ts	Cliente HTTP centralizado con fetch.
frontend/src/app.ts	Eventos, estado de ítems y manipulación del DOM.
static/index.html	Estructura de las tres secciones.
static/css/styles.css	Diseño minimalista y responsive.
src/test	Pruebas de controlador, servicio, consulta y exportaciones.

Por qué hay TypeScript y JavaScript

El navegador ejecuta JavaScript, no TypeScript. Por eso se entregan los .ts legibles para desarrollar y los .js compilados dentro de static/js para ejecutar. TypeScript ayuda a detectar campos mal escritos antes de abrir la página.

Comandos

```
# Compilar el frontend cuando cambian los .ts
cd frontend && npm install && npm run build

# Ejecutar pruebas y levantar el servidor
mvn test
mvn spring-boot:run

# Abrir en el navegador
http://localhost:8080
```

6. Guion exacto para la demo

Preparación antes de entrar al aula

- Confirmar que PostgreSQL está iniciado y existe la base facturacion_orm.
- Ejecutar mvn test y mostrar que todas las pruebas pasan.
- Ejecutar mvn spring-boot:run y esperar Started FacturaArcaApplication.
- Abrir http://localhost:8080 y verificar que aparezcan los catálogos y comprobantes.

Recorrido sugerido (3 a 5 minutos)

Paso	Acción visible	Qué explicar
1	Mostrar el listado inicial.	GET trae DTOs resumidos sin exponer entidades.
2	Filtrar por EMITIDA.	TypeScript arma query params y actualiza la tabla.
3	Completar número, fecha y cabecera.	Los selectores usan IDs reales de /api/catalogos.
4	Agregar dos renglones y una bonificación.	El navegador previsualiza; el backend vuelve a calcular.
5	Presionar Emitir factura.	POST devuelve 201 y JPA guarda detalles por cascada.
6	Buscar la factura recién creada.	La respuesta vuelve de PostgreSQL y se renderiza.
7	Presionar Anular.	PUT cambia estado y fechaAnulacion dentro de una transacción.
8	Opcional: eliminar una factura de prueba.	DELETE devuelve 204; aclarar que es eliminación física.

Frase para explicar la seguridad del cálculo

> **“Aunque el frontend muestra una vista previa, el total confiable se calcula otra vez en el backend con el precio guardado en la base. El cliente no puede inventar el precio final.”**

Si algo no carga

- Revisar primero la terminal de Spring Boot: debe indicar Tomcat started on port 8080.
- Si faltan selectores, comprobar que la base tenga Usuario, Cliente, Artículo y ListaPrecioArticulo.
- Si el puerto está ocupado, cerrar la instancia anterior antes de volver a ejecutar.
- No abrir index.html como archivo suelto: siempre entrar por http://localhost:8080.

7. Preguntas que puede hacer el profesor

Pregunta	Respuesta corta
¿Por qué DTOs?	Para desacoplar el JSON de las entidades y controlar exactamente qué entra y qué sale.
¿Por qué mandar IDs?	Evita recibir grafos de entidades y permite validar cada relación en el servidor.
¿Qué hace @Transactional?	Garantiza todo o nada: si falla un detalle, se revierte la factura completa.
¿Cómo se guardan los detalles?	La relación OneToMany tiene cascade ALL y cada detalle referencia su factura.
¿Qué significa orphanRemoval?	Un detalle quitado de la colección se elimina también de la base.
¿Dónde se calcula el IVA?	En FacturaService, usando 21% sobre el neto luego de la bonificación.
¿Qué diferencia hay entre PUT y POST?	POST crea un recurso nuevo; PUT actualiza uno identificado por su ID.
¿Por qué 201, 200 y 204?	201 para creación, 200 cuando hay respuesta y 204 al borrar sin cuerpo.
¿Cómo devuelven 404?	RecursoNoEncontradoException es traducida por ApiExceptionHandler.
¿Qué hace fetch?	Envía solicitudes HTTP desde TypeScript y devuelve una Promise.
¿Es una aplicación web?	Sí: navegador como cliente, servidor Spring Boot y PostgreSQL como persistencia.
¿Es Spring?	Usa el ecosistema Spring mediante Spring Boot 3.

Idea central para cerrar

> **El proyecto está separado por responsabilidades: la pantalla captura, el controlador comunica, el servicio decide y calcula, y los repositorios persisten. Esa separación hace que el código sea más fácil de probar, mantener y explicar.**

Checklist final

- Frontend compilado sin errores de TypeScript.
- Pruebas Maven aprobadas.
- Servidor conectado a PostgreSQL.
- Creación, filtrado, anulación y eliminación verificados.
- README reducido a los integrantes, según lo pedido.